Type hints, notebooks, and reproducibility in Python

PART ONE	PART TWO	PART THREE
Type hints and clean-code practices	**Notebook versus production code**	**Reproducibility practices**

This document covers three habits that decide whether Python work survives contact with other people: annotating your code so its contracts are checkable, moving analysis out of notebooks and into software, and making results repeatable by someone who was not there when you produced them. Each part is written to be read on its own, with runnable examples and a checklist at the end.

1. Type hints and clean-code practices

1.1 What type hints are, and what they are not

A type hint is an annotation attached to a variable, parameter, or return value. Python stores annotations but does not enforce them: nothing at runtime stops you from passing a string where an `int` is declared. Their value comes from the tools that read them — type checkers (mypy, pyright/Pylance, ty), editors (completion, go-to-definition, inline errors), and libraries that use annotations as a schema at runtime (pydantic, FastAPI, dataclasses, attrs, typer, SQLAlchemy 2.0).

So annotations buy you three things. Errors surface at edit time rather than in production. The signature becomes documentation that cannot silently rot, because the checker fails when it disagrees with the code. And refactors become mechanical: rename a field, run the checker, fix what it lists.

> THE ONE-LINE RULE
>
> Annotate every boundary — public functions, class attributes, and module-level constants. Leave obvious local variables alone; the checker infers them.

1.2 The syntax, from basic to complete

Annotations are additive: the same code runs identically with or without them. What changes is what a reader and a checker can know. The pairs below are the whole vocabulary — on the left the code as it is usually first written, on the right the same code annotated.

BEFORE — UNTYPED	AFTER — ANNOTATED
`MAX_RETRIES = 3`	`MAX_RETRIES: int = 3`
`rate = 0.075`	`rate: float = 0.075`
`scores = []`	`scores: list[float] = []`
`index = {}`	`index: dict[str, list[int]] = {}`
`pair = ("gross", 3)`	`pair: tuple[str, int] = ("gross", 3)`

`def net(gross, rate=0.2):`
`  return gross * (1 - rate)`
	`def net(`
	`  gross: float, rate: float = 0.2`
	`) -> float:`
	`  return gross * (1 - rate)`

`def find_user(uid):`
`  ...`
	`def find_user(`
	`  uid: int,`
	`) -> User | None:`
	`  ...`

`def resize(img, width=None):`
`  ...`
	`def resize(`
	`  img: Image,`
	`  width: int | None = None,`
	`) -> Image:`
	`  ...`

`def save(path, rows):`
`  ...`
	`def save(`
	`  path: Path, rows: Sequence[Row]`
	`) -> None:`
	`  ...`

`MODE = "train"`	`Mode = Literal["train", "eval"]` `MODE: Final[Mode] = "train"`

`config = {}`	`config: TrainConfig = TrainConfig(`
	`  dataset="q3",`
	`)`

`def total(values):`
`  return sum(values)`
	`def total(`
	`  values: Iterable[float],`
	`) -> float:`
	`  return sum(values)`

Figure 1 — The same declarations, before and after, paired row by row. Nothing about the runtime behaviour differs; every question a reader would otherwise answer by reading the function body is now answered in the signature.

The rest of this section takes those constructs one at a time.

```
from decimal import Decimal

def net(gross: Decimal, rate: float = 0.2) -> Decimal:
    """Return the amount after tax."""
    return gross * Decimal(1 - rate)

def log_event(
    name: str, /, *tags: str, level: str = "info", **fields: object
) -> None:
    ...            # *args -> each tag is str
                   # **kwargs -> each field value is object
```

Figure 2 — Parameters, defaults, variadics, and a `None` return. A function returning nothing is annotated `-> None`, not left blank.

Containers. Since Python 3.9 you annotate with the builtin generics directly; the `typing.List` family is deprecated legacy. Annotate what is inside the container, not just the container.

```
scores: list[float]
index: dict[str, list[int]]
pair: tuple[str, int]                    # exactly two items, fixed types
row: tuple[str, ...]                     # any length, all str
seen: set[bytes]
frozen: frozenset[str]
```

Figure 3 — Builtin generic containers.

Optionality and unions. `X | None` (3.10+) replaces `Optional[X]`, and `A | B` replaces `Union[A, B]`. A parameter defaulting to `None` must say so in its type — the default does not widen it for you.

```
def find_user(uid: int) -> User | None: ...

def resize(img: Image, width: int | None = None) -> Image: ...

user = find_user(7)
print(user.email)          # error: "None" has no attribute "email"

if user is not None:       # narrowing: inside the branch, user is User
    print(user.email)
```

Figure 4 — Optional types force you to handle the empty case. This is where checkers pay for themselves.

Narrow parameters, wide returns. Accept the most general type you can actually work with and return the most specific one you have. If a function only iterates, take `Iterable[str]`; if it also indexes, take `Sequence[str]`; take `list[str]` only if you mutate the caller's list. Use `Mapping` instead of `dict` for read-only parameters — the immutable protocols are covariant, so `Mapping[str, object]` accepts a `dict[str, int]` while `dict[str, object]` does not.

```
from collections.abc import Iterable, Iterator, Mapping, Sequence, Callable

def total(values: Iterable[float]) → float:          # only iterates
    return sum(values)

def head(rows: Sequence[str], n: int = 5) → list[str]:   # indexes/slices
    return list(rows[:n])

def merge(
    base: Mapping[str, str], extra: Mapping[str, str]
) → dict[str, str]:
    return {**base, **extra}

def chunks(data: Sequence[int], size: int) → Iterator[list[int]]:
    for i in range(0, len(data), size):
        yield list(data[i : i + size])               # generator → Iterator[T]

Validator = Callable[[str], bool]                    # takes str, returns bool
AnyHandler = Callable[..., None]                     # any arguments
```

Figure 5 — Import abstract containers from `collections.abc`, not `typing`.

Aliases, literals, and constants. Aliases name a shape you repeat; `Literal` constrains a value to a fixed set, which is how you replace stringly-typed flags; `Final` marks a binding that must not be reassigned.

```
from typing import Final, Literal

type FeatureRow = dict[str, float]        # 3.12+ alias statement
type Splits = tuple[FeatureRow, FeatureRow]

Mode = Literal["train", "eval", "predict"]
DEFAULT_MODE: Final[Mode] = "train"

def run(mode: Mode) → None: ...
run("trian")                              # caught at check time, not 3am
```

Figure 6 — On Python 3.9–3.11 write `FeatureRow: TypeAlias = dict[str, float]` instead of the `type` statement.

1.3 Structuring data: dataclass, TypedDict, NamedTuple, pydantic

A large share of unclear Python is dictionaries passed between functions with no record of their keys. Give the structure a name and the checker can help.

```
from dataclasses import dataclass, field
from typing import TypedDict, NotRequired, NamedTuple

@dataclass(frozen=True, slots=True)          # immutable, low memory
class TrainConfig:
    dataset: str
    lr: float = 1e-3
    epochs: int = 10
    tags: list[str] = field(default_factory=list)   # never `tags: list = []`

class ApiPayload(TypedDict):            # keys can't be misspelled
    user_id: int
    email: str
    referrer: NotRequired[str]

class Point(NamedTuple):                     # immutable, tuple-compatible
    x: float
    y: float
```

Figure 7 — Three ways to name a shape.

USE	WHEN	VALIDATES AT RUNTIME
`@dataclass`	Internal objects you construct in your own code	No
`TypedDict`	JSON-shaped dicts you must keep as dicts	No
`NamedTuple`	Small immutable records, tuple unpacking	No
`pydantic.BaseModel`	Untrusted input: API bodies, YAML config, CSV rows	Yes

Table 1 — Choosing a container for structured data. At the edges of the system, validate; inside it, plain dataclasses are enough.

```
from pathlib import Path
import yaml
from pydantic import BaseModel, Field

class Settings(BaseModel):
    data_path: Path
    lr: float = Field(gt=0, le=1)
    epochs: int = Field(ge=1, le=1000)
    seed: int = 0

def load_settings(path: Path) -> Settings:
    return Settings.model_validate(yaml.safe_load(path.read_text()))
```

Figure 8 — A validated config. A bad YAML file now fails at startup with a precise message instead of producing a silently wrong run.

1.4 Generics, protocols, and the rest of the toolkit

Generics let a function's return type depend on its input type. Python 3.12 introduced the inline parameter syntax; before that you declared a `TypeVar`.

```
def first[T](items: Sequence[T]) -> T | None:        # 3.12+
    return items[0] if items else None

class Cache[K, V]:
    def __init__(self) -> None:
        self._data: dict[K, V] = {}
    def get(self, key: K) -> V | None:
        return self._data.get(key)

# Pre-3.12 equivalent
from typing import TypeVar
T = TypeVar("T")
def first_legacy(items: Sequence[T]) -> T | None: ...
```

Figure 9 — `first(["a"])` is now known to return `str | None`, not `Any`.

Protocols express structural typing: "anything with these methods". They are how you type duck-typed interfaces without forcing callers to inherit from your base class, and they are the cleanest way to make code testable — a fake that satisfies the protocol type-checks without any mocking framework.

```
from typing import Protocol, runtime_checkable

class Store(Protocol):
    def read(self, key: str) → bytes: ...
    def write(self, key: str, blob: bytes) → None: ...

def checkpoint(store: Store, state: bytes) → None:   # accepts S3Store,
    store.write("latest", state)                     # LocalStore, FakeStore

class FakeStore:                            # no inheritance needed
    def __init__(self) → None: self.data: dict[str, bytes] = {}
    def read(self, key: str) → bytes: return self.data[key]
    def write(self, key: str, blob: bytes) → None: self.data[key] = blob
```

Figure 10 — Depend on a protocol, not a concrete class. This is dependency inversion with no framework.

Overloads describe functions whose return type depends on argument values — common in data code where a flag switches the output shape. **`Self`** types fluent builders and `from_*` constructors. **`Never`** marks a function that never returns.

```
from typing import Literal, NoReturn, Self, overload

@overload
def load(path: Path, *, lazy: Literal[True]) → Iterator[dict[str, str]]: ...
@overload
def load(
    path: Path, *, lazy: Literal[False] = False
) → list[dict[str, str]]: ...
def load(path: Path, *, lazy: bool = False):     # implementation
    rows = (json.loads(line) for line in path.open())
    return rows if lazy else list(rows)

class Query:
    def where(self, **kw: object) → Self: ...      # subclasses stay subclasses

def fail(msg: str) → NoReturn:
    raise RuntimeError(msg)
```

Figure 11 — Overloads, `Self`, and `NoReturn`.

Escape hatches, used deliberately. `Any` disables checking for a value and spreads through everything it touches — prefer `object` when you mean "some value I won't touch", since `object` still forces a narrowing check before use. `cast(T, x)` asserts a type without a runtime check; keep it to one line with a comment saying why the checker cannot see it. #

`type: ignore[code]` should always carry the specific error code so it stops suppressing everything else. For third-party libraries with no stubs, install the `types-*` package if it exists and otherwise confine the untyped surface to one thin wrapper module.

Array and dataframe code is annotated coarsely. `numpy.typing.NDArray[np.float64]` captures dtype but not shape; write the shape in the docstring. Pandas has `pandas-stubs` but a `DataFrame` annotation says nothing about columns, so validate schemas at runtime with pandera or a small assertion helper at each stage boundary.

```
import numpy as np
import numpy.typing as npt
import pandas as pd

def standardise(x: npt.NDArray[np.float64]) -> npt.NDArray[np.float64]:
    """x: (n_samples, n_features). Returns the same shape, zero mean."""
    return (x - x.mean(axis=0)) / x.std(axis=0)

REQUIRED = {"user_id", "ts", "amount"}

def check_columns(df: pd.DataFrame) -> pd.DataFrame:
    missing = REQUIRED - set(df.columns)
    if missing:
        raise ValueError(f"missing columns: {sorted(missing)}")
    return df
```

Figure 12 — Types where they work, runtime checks where they don't.

1.5 Adopting types in an existing codebase

Do not annotate everything at once. Turn the checker on in permissive mode, make it a required CI check so no new violations land, then tighten module by module — starting with the code most depended upon, since types on a leaf function help one caller while types on a core utility help everyone.

```
# pyproject.toml
[tool.mypy]
python_version = "3.12"
strict = true                       # the target
warn_unused_ignores = true
warn_redundant_casts = true

[[tool.mypy.overrides]]             # legacy corners, temporarily exempt
module = ["legacy.*", "scripts.*"]
disallow_untyped_defs = false

[tool.ruff]
line-length = 100
[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM", "ANN", "RUF"]
ignore = ["ANN101"]
```

Figure 13 — A gradual-typing configuration. `strict = true` globally, with named exemptions you delete over time.

1.6 Clean-code practices that types cannot give you

Names. Functions are verbs, variables are nouns, booleans read as predicates (`is_valid`, `has_expired`). No single letters outside tight numeric loops, no abbreviations a newcomer must decode, no `data`/`result`/`tmp` as the name of anything that lives more than three lines. Length should scale with scope: `i` inside a comprehension is fine, a module-level `df` is not.

Functions. One job each. If you cannot name it without "and", split it. Keep them short enough to read without scrolling and keep the parameter list to about four positional arguments; past that, group them into a dataclass or make them keyword-only with `*`. Return early with guard clauses instead of nesting — flat code with three early returns beats a four-level pyramid.

```
# Before: nested, does three things, mutable default
def process(rows, cfg={}, log=[]):
    if rows:
        if cfg.get("clean"):
            rows = [r for r in rows if r["amount"] > 0]
            if cfg.get("scale"):
                rows = [{**r, "amount": r["amount"] * cfg["factor"]}
                        for r in rows]
        log.append("done")
        return rows

# After: guard clauses, one job per function, immutable inputs
def drop_non_positive(rows: Sequence[Row]) -> list[Row]:
    return [r for r in rows if r["amount"] > 0]

def scale_amounts(rows: Sequence[Row], factor: float) -> list[Row]:
    return [{**r, "amount": r["amount"] * factor} for r in rows]

def process(rows: Sequence[Row], cfg: CleanConfig) -> list[Row]:
    if not rows:
        return []
    cleaned = drop_non_positive(rows) if cfg.clean else list(rows)
    return scale_amounts(cleaned, cfg.factor) if cfg.scale else cleaned
```

Figure 14 — The refactor covers four defects at once: the mutable default arguments (`cfg={}`, `log=[]` are created once and shared across calls), the implicit `None` return, the nesting, and the untyped config.

Purity and side effects. Keep computation separate from I/O. A function that reads a file, transforms it, and writes a database row cannot be tested without a filesystem and a database; three functions can. Push I/O to the edges — read at the start, compute in the middle, write at the end — and pass dependencies in as arguments rather than importing a global client inside the function body.

Errors. Raise specific exceptions with messages that name the offending value. Never write a bare `except:` or a silent `except Exception: pass`; catch the narrowest exception you can handle, and if you re-raise a wrapped error use `raise NewError(...) from exc` to keep the chain. Validate inputs at the boundary so internal functions can assume valid data.

Logging, not printing. Use the `logging` module with a module-level `logger = logging.getLogger(__name__)`, configured once at the entry point. Log structured context (ids, counts, durations), never secrets. `print` belongs in a CLI's user-facing output and nowhere else.

Configuration. No magic numbers, no hard-coded paths, no credentials in source. Constants go at module top in `UPPER_CASE`; anything environment-specific comes from a config file or environment variable, read once into a validated settings object (Figure 8). Paths use `pathlib.Path`, not string concatenation.

Docstrings and comments. Every public function gets a docstring saying what it returns and what it raises; with types in the signature you do not repeat the types in prose. Comments explain *why* — the constraint, the ticket, the paper — because the code already says what. Delete commented-out code; git remembers it.

Automate the rest. Formatting and lint arguments are wasted time: run `ruff format` and `ruff check --fix` in a pre-commit hook and again in CI, so no review ever discusses whitespace.

TOOL	JOB	RUNS IN
ruff format	Formatting (Black-compatible)	pre-commit, CI
ruff check	Lint, import sort, modernisation	pre-commit, CI
mypy / pyright	Static type checking	editor, CI
pytest	Tests, fixtures, parametrisation	CI
pre-commit	Runs the above on staged files	git hook

Table 2 — A minimal toolchain. Every entry is a decision you no longer have to make by hand.

2. Notebook versus production code

Notebooks and modules are not competing styles of the same activity. A notebook is an instrument for exploration: fast feedback, inline plots, a narrative you can hand to a colleague. Production code is an instrument for repetition: it must run unattended, produce the same answer twice, and be safe for someone else to change. The failure mode in most teams is not choosing a side but leaving exploration code in the notebook long after it became infrastructure.

2.1 What breaks when a notebook becomes infrastructure

PROBLEM	WHY IT BITES
Hidden state	The kernel holds variables whose defining cell was edited or deleted. The notebook works on your machine and nowhere else.
Out-of-order execution	Execution counts say `[12]` then `[7]`. The result cannot be reproduced by running top to bottom.
No tests	Logic lives in cells, which nothing can import or assert against.
Unreviewable diffs	`.ipynb` is JSON with embedded outputs and base64 images; git diffs are noise and merges conflict constantly.
Copy-paste duplication	The same cleaning block exists in six notebooks with three different bug fixes applied.
Leaked secrets and outputs	Committed outputs carry credentials, tokens, customer rows, and PII into the repository history.
No entry point	Nothing to schedule, containerise, parameterise, or call from another service.

Table 3 — The recurring failure modes.

2.2 Drawing the boundary

Keep in the notebook: loading a sample, looking at distributions, trying a model, plotting, and the written narrative of what you found. Move out of the notebook the moment code is *reused* (a second notebook imports it), *scheduled* (it runs without you), or *relied upon* (a decision or a downstream system depends on its output). A mature repository has thin notebooks that import from a package and mostly contain calls and charts.

```
project/
├── pyproject.toml           # deps, tool config, package metadata
├── uv.lock                  # exact resolved versions
├── src/
│   └── churn/
│       ├── __init__.py
│       ├── config.py        # validated Settings object
│       ├── data.py          # load / clean / split      (pure, tested)
│       ├── features.py      # feature engineering       (pure, tested)
│       ├── model.py         # train / evaluate / persist
│       └── cli.py           # `churn train --config conf/base.yaml`
├── tests/
│   ├── test_data.py
│   └── test_features.py
├── notebooks/
│   └── 2026-08-eda.ipynb    # imports churn.*, holds narrative + plots
├── conf/base.yaml
└── README.md
```

Figure 15 — The `src/` layout. Code is installed (`uv pip install -e .`) so notebooks, tests, and the scheduler all import the same package.

2.3 The migration, step by step

1. **Make it run top to bottom.** Restart the kernel and run all. Fix what breaks. Until this passes, you do not know what the notebook actually does.
2. **Group cells into stages.** Load, clean, feature, train, evaluate, report. Insert markdown headers so the structure is visible.
3. **Turn each stage into a function** in the notebook first — parameters in, value out, no reliance on globals. This is where hidden dependencies surface.
4. **Move the functions into a module** and import them back. The notebook shrinks to a sequence of calls; behaviour should be unchanged.
5. **Lift constants into config.** Paths, dates, thresholds, hyperparameters go to a YAML file loaded into a validated settings object.
6. **Write tests as you go.** One test per moved function, using a tiny fixture dataframe committed to the repo — not a sample of production data.
7. **Add a CLI entry point** so the pipeline runs headless with an explicit config, and log instead of printing.
8. **Delete the dead cells** and keep the notebook as the exploration record it is.

```
# src/churn/cli.py
import logging
from pathlib import Path
import typer
from churn.config import load_settings
from churn.data import load_raw, clean, split
from churn.model import train, evaluate

app = typer.Typer(add_completion=False)
log = logging.getLogger(__name__)

@app.command()
def run(config: Path, out: Path = Path("artifacts")) → None:
    """Train and evaluate one model from a config file."""
    logging.basicConfig(
        level=logging.INFO, format="%(asctime)s %(name)s %(message)s"
    )
    cfg = load_settings(config)
    train_df, test_df = split(clean(load_raw(cfg.data_path)), seed=cfg.seed)
    model = train(train_df, cfg)
    metrics = evaluate(model, test_df)
    log.info("auc=%.4f n_test=%d", metrics.auc, len(test_df))
    (out / "metrics.json").write_text(metrics.model_dump_json(indent=2))

if __name__ == "__main__":
    app()
```

Figure 16 — The same pipeline as a headless command. Everything variable arrives through `config`; nothing is read from a global.

2.4 Making notebooks behave in the meantime

Notebooks can be brought most of the way to respectability with tooling, and you should apply it even to exploration.

- **Strip outputs before committing** — `nbstripout` as a git filter, or store notebooks as paired `.py` files with **jupytext** so review happens on plain text.
- **Diff and merge with nbdime** when you must keep `.ipynb` in git.
- **Parameterise and execute with papermill** for scheduled notebook runs, so inputs are explicit and the executed copy is an artifact.
- **Test them in CI** — `pytest --nbmake notebooks/` at minimum proves they still run top to bottom.
- **Lint them** — `ruff check` reads `.ipynb` directly; `nbqa` wraps other tools.

- **Never `%autoreload` into a deliverable.** It is fine while editing a module, but re-run with a fresh kernel before you trust a number.
- **Name notebooks by date and topic** (`2026-08-churn-eda.ipynb`) and keep one owner per notebook.

> TEST FOR READINESS
>
> Can a colleague clone the repository, run one documented command, and get your number — without opening a notebook or asking you a question? If not, the work is still exploration, whatever it is being used for.

3. Reproducibility practices

Reproducibility means a stated result can be regenerated from recorded inputs. It is not one switch but a stack of four independent things: the same **code**, the same **environment**, the same **data**, and the same **randomness**. Fail any one and the result becomes a story rather than a fact. It is worth being explicit about the levels — *repeatable* (same machine, same result today and next month), *reproducible* (another person, another machine, same result), and *replicable* (a different implementation reaches the same conclusion). Most teams need the middle one and accidentally aim for the first.

3.1 Code: one commit per result

Every number you report should be attributable to a commit. That means committing before every run that matters, refusing to run pipelines from a dirty working tree, and recording the commit hash in the output alongside the metrics. Tag or record the exact revision in the artifact, not in a chat message.

```
import json, platform, subprocess, sys
from datetime import datetime, timezone
from pathlib import Path

def git_state() → dict[str, str | bool]:
    cmd = ["git", "rev-parse", "HEAD"]
    sha = subprocess.check_output(cmd, text=True).strip()
    status = subprocess.check_output(
        ["git", "status", "--porcelain"], text=True
    )
    return {"commit": sha, "dirty": bool(status)}

def write_run_manifest(
    path: Path,
    config: dict[str, object],
    metrics: dict[str, float],
) → None:
    path.write_text(json.dumps({
        "git": git_state(),
        "python": sys.version,
        "platform": platform.platform(),
        "created_utc": datetime.now(timezone.utc).isoformat(),
        "config": config,
        "metrics": metrics,
    }, indent=2, sort_keys=True))
```

Figure 17 — A run manifest written beside every artifact. `dirty: true` is the single most useful field in the file.

3.2 Environment: pin, lock, and isolate

Declaring `pandas` in `requirements.txt` is not pinning; it is a promise to install whatever exists on install day, including transitive dependencies you never named. Three layers, each stricter than the last:

- **Declared ranges** in `pyproject.toml` — what your code needs (`pandas>=2.2,<3`). Pin the Python version too, in `requires-python` and a `.python-version` file.
- **A lockfile** committed to git — every package, every transitive dependency, exact version and hash. Produced by `uv lock`, `poetry lock`, or `pip-compile`. This is the artifact that makes an install reproducible; install from it with `uv sync --frozen`.
- **A container image** when system libraries matter (CUDA, GDAL, a compiler, a database client). Build from the lockfile so the image and the repository cannot diverge, and pin the base image by digest, not by `:latest`.

```
# create and lock
uv venv --python 3.12
uv add pandas scikit-learn pydantic
uv lock                       # writes uv.lock  → commit this

# reproduce, months later, on another machine
uv sync --frozen              # installs exactly the locked versions
uv run churn train --config conf/base.yaml
```

Figure 18 — Lock, commit, and always run through the locked environment. Never `pip install` into a project environment by hand; if it is not in the manifest, it does not exist.

3.3 Randomness: seed every source

Setting `random.seed(0)` is rarely enough, because numpy, the framework, the hash function, and each data-loader worker have their own generators. Seed all of them from one configured value, and prefer explicit generator objects over global state so parallel code stays deterministic.

```
import os, random
import numpy as np

def seed_everything(seed: int) → np.random.Generator:
    os.environ["PYTHONHASHSEED"] = str(seed)  # set before launch
    random.seed(seed)
    np.random.seed(seed)                  # legacy global, for libs
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.use_deterministic_algorithms(True, warn_only=True)
        torch.backends.cudnn.benchmark = False   # autotuner varies
    except ImportError:
        pass
    return np.random.default_rng(seed)    # pass this explicitly

rng = seed_everything(0)
sample = rng.choice(1_000, size=10, replace=False)
```

Figure 19 — Note that PYTHONHASHSEED only takes effect if set before the interpreter starts, so set it in the launcher or Dockerfile as well.

Some nondeterminism is not fixable by seeding, and you should know which: GPU floating-point reductions are order-dependent, `set` and `dict` iteration over hashed objects can vary, thread and process scheduling changes result order, and any use of the wall clock or `uuid4` makes output differ by definition. Sort before you aggregate, pass timestamps in as parameters instead of calling `datetime.now()` deep inside a function, and derive ids deterministically where you can.

3.4 Data: immutable inputs and deterministic splits

Treat raw data as read-only. Never edit it in place, never overwrite a snapshot, and never point a pipeline at a live table that changes under it. Write derived data to new paths keyed by version, and record a checksum so you can prove which bytes produced a result.

```
import hashlib
from pathlib import Path
import pandas as pd

def sha256(path: Path, chunk: int = 1 << 20) → str:
    h = hashlib.sha256()
    with path.open("rb") as fh:
        for block in iter(lambda: fh.read(chunk), b""):
            h.update(block)
    return h.hexdigest()

def stable_split(
    df: pd.DataFrame, key: str, test_frac: float = 0.2
) → tuple[pd.DataFrame, pd.DataFrame]:
    """Hash split: a row's side never changes as data grows."""
    digest = df[key].astype(str).map(
        lambda k: int(hashlib.md5(k.encode()).hexdigest()[:8], 16)
        / 0xFFFFFFFF
    )
    is_test = digest < test_frac
    return df[~is_test].copy(), df[is_test].copy()
```

Figure 20 — Hashing the key rather than shuffling means yesterday's test set is still the test set after new rows arrive — a common silent source of leakage and of unreproducible metrics.

For larger datasets, version them the way you version code: DVC, lakeFS, or a table format with snapshots (Delta, Iceberg) let a config record a dataset version that can be checked out later. When the source is a warehouse, capture the query *and* a snapshot timestamp or partition bound, so "as of" is part of the recorded input rather than an accident of when you ran it.

3.5 Recording runs

A pipeline that cannot be re-run from its own record is not reproducible even if every ingredient is pinned. Log for every run: the config (whole file, not a summary), the code commit, the environment lock hash, the input data version and checksums, the seed, the metrics, and the output artifact paths. Tools like MLflow, Weights & Biases, or DVC experiments do this for you; a JSON manifest per run in object storage (Figure 17) is a legitimate minimum. The rule to enforce is that the config file, not the command line and not a notebook cell, is the complete description of a run.

Finally, test reproducibility rather than assuming it. A cheap regression test — fixed seed, small fixture, assert the metric to a tolerance — catches the day a library upgrade silently changes your numbers, which is exactly the day you would otherwise not notice.

```
# tests/test_reproducible.py
import pytest
from churn.data import stable_split
from churn.model import train, evaluate

def test_metric_is_stable(fixture_df):
    train_df, test_df = stable_split(fixture_df, key="user_id")
    a = evaluate(train(train_df, seed=0), test_df).auc
    b = evaluate(train(train_df, seed=0), test_df).auc
    assert a == pytest.approx(b, abs=1e-9)      # same seed, same answer
    assert a == pytest.approx(0.8123, abs=1e-3)  # golden value
```

Figure 21 — Two assertions: determinism within a version, and a golden value across versions.

Checklists

TYPES AND CLEAN CODE

- Every public function has annotated parameters and return type.
- No bare `dict`/`list`/`tuple`; container contents are annotated.
- Optional values are `X | None` and every caller handles the empty case.
- Parameters take the narrowest protocol that works (`Iterable`, `Mapping`, `Sequence`).
- Repeated dict shapes are dataclasses, TypedDicts, or pydantic models.
- Untrusted input is validated at the boundary; internals assume valid data.
- No mutable default arguments, no bare `except`, no `print` for logging.
- No magic numbers, hard-coded paths, or secrets in source.
- Functions do one thing, use guard clauses, and separate computation from I/O.
- `ruff`, the type checker, and `pytest` run in pre-commit and CI, and block merges.

NOTEBOOK TO PRODUCTION

- The notebook runs clean from a restarted kernel, top to bottom.
- Reusable logic lives in an installed package under `src/`, imported by the notebook.
- Every moved function has at least one test against a committed fixture.
- Constants live in a config file loaded into a validated settings object.
- A CLI entry point runs the whole pipeline headless from that config.
- Outputs are stripped (or the notebook is paired to `.py`) before commit.
- Notebooks are named by date and topic, and dead cells are deleted.

REPRODUCIBILITY

- A lockfile and a pinned Python version are committed; installs use `--frozen`.
- Runs are refused, or flagged, from a dirty working tree.
- One seed value from config seeds Python, numpy, the framework, and the hash seed.
- Raw data is read-only; derived data is written to versioned paths with checksums.
- Splits are deterministic under dataset growth (hash the key, don't shuffle).
- No wall-clock calls or unsorted aggregations inside pipeline logic.
- Every run writes a manifest: commit, config, data version, seed, metrics, artifacts.
- A regression test asserts the headline metric against a golden value.